Firmware V-GUARD — decisões para aprovação

Oct 13, 2026 · @Pedro

O planejamento do firmware do ESP32 está fechado. Só três pontos dependem da confirmação da Isabela; o resto desta página é informativo. O mais importante para todos: o ESP32 vai fazer requisições OBD-II padronizadas ao carro, com salvaguardas, em vez de só escutar o barramento.

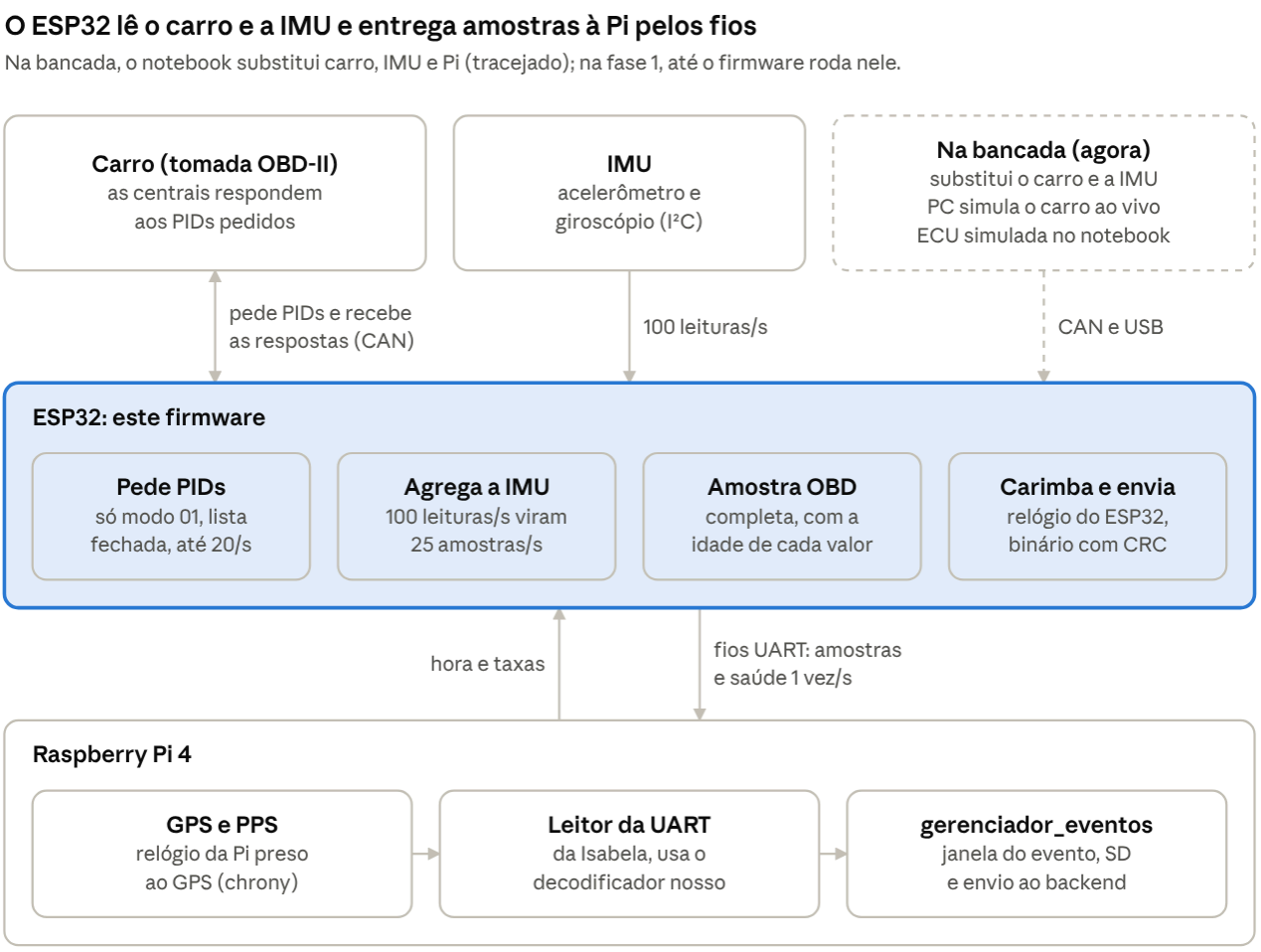
Para a Isabela confirmar

Três pontos mudam o código do gerenciador_eventos. Marque cada um quando concordar, ou comente nele.

- [ ] **Leitor da UART (D10):** você fica com o leitor na Pi, usando o decodificador que entregamos. Ele é em Python, só com a biblioteca padrão, valida o CRC, converte o horário e devolve dicionários prontos. Vem com especificação, gravações de teste e um guia.
- [ ] **Formato da amostra OBD (D5):** a amostra chega sempre completa, porque o `update("obd", ...)` substitui a anterior inteira; um valor por vez faria o evento sair sem RPM. O formato é `{"rpm": 2100, "speed": 60, "age_ms": {"rpm": 120, "speed": 80}}`, e com o carro desligado os valores continuam, com a idade crescendo.
- [ ] **Janela do evento no 4G (D4):** a 25 amostras/s de IMU, cada janela de 20 s soma cerca de 100 kB só de IMU, em JSON sem compressão. Ela precisa subir inteira pelo 4G, ou pode ter a taxa reduzida só nesse envio?

Também é bom saber: o `ts` de cada amostra já chega no relógio da Pi (D12), perdas são contadas e nunca reenviadas (D11), e as taxas de envio mudam por uma chamada do decodificador (D7).

Como o firmware se encaixa



arquitetura do firmware · carro, ESP32 e Pi

O ESP32 só conversa com o carro e com a Pi. Eventos, armazenamento e envio ao backend continuam na Pi, como já estão no gerenciador_eventos.

As quatro fases



as quatro fases · do notebook à integração

Na fase 1, uma camada fina troca o FreeRTOS, o CAN e a UART do ESP32 por equivalentes do Linux. Nas fases seguintes, só essa camada muda; a lógica testada no notebook é a mesma.

Atenção: o ESP32 vai perguntar ao carro

O OBD-II funciona por pergunta e resposta. Se o ESP32 só escutasse o barramento, o projeto ficaria sem RPM, velocidade e demais dados do carro (RF03 a RF05).

- Para ler o RPM, alguém envia "modo 01, PID 0x0C" e a central do motor responde. Sem perguntar, não há resposta.
- Em muitos carros o conector OBD fica mudo até alguém perguntar, por causa de um gateway. O tráfego espontâneo, quando existe, é proprietário de cada montadora e está fora do escopo pelo AE04.
- O Plano de Projeto já permite as requisições padronizadas de diagnóstico (AE02 e RNF10). É o que qualquer scanner de oficina ou adaptador ELM327 faz.

O risco real está em outros serviços do diagnóstico: apagar códigos de falha, escrever na memória da central, acionar atuadores. O firmware se protege assim:

- **Lista fechada:** só passam requisições de leitura do modo 01 para os endereços padronizados. Qualquer outro quadro é bloqueado, contado e coberto por teste automático.
- **Escutar antes de falar:** ao conectar, o ESP32 só escuta e não transmite se perceber erros no barramento.
- **Velocidade fixa de 500 kbit/s:** o ESP32 nunca testa velocidades desconhecidas. Se a primeira requisição gerar erro, ele para.

- **Ritmo limitado:** no máximo 20 requisições por segundo, e nenhuma enquanto o carro não responde (ignição desligada), com nova tentativa a cada 5 s.
- **Em estudo:** um transceptor com modo silencioso, que impede transmissões no próprio hardware mesmo se o ESP32 reiniciar ou travar.

Decisões já tomadas

Estas decisões já valem e estão aqui para informação. A coluna "Interessa a" diz quem deve ler cada uma.

#	Decisão	Por quê	Interessa a
D1	O ESP32 faz requisições OBD-II padronizadas (modo 01) ao carro, em vez de só escutar.	Sem perguntar não há RPM nem velocidade; AE02 e RNF10 já permitem.	Todos
D2	Salvaguardas no CAN: lista fechada, escutar antes de falar, até 20 requisições/s, pausa quando o carro não responde.	Bloqueiam os serviços de diagnóstico perigosos, com teste automático.	Nicolas
D3	Barramento fixo em 500 kbit/s com IDs de 11 bits; muda recompilando após o T17.3.	O ESP32 nunca transmite numa velocidade desconhecida.	Nicolas
D4	A IMU é lida 100 vezes/s; a Pi recebe 25 amostras/s com média e pico.	Janela do evento menor no 4G; o pico preserva impactos.	Isabela
D5	Amostra OBD completa, com a idade de cada valor.	O update() da Pi substitui a amostra inteira.	Isabela
D6	PIDs fixos: RPM, velocidade e os seis do RF05, só os que o carro suportar.	Evita tráfego desnecessário (RNF10); pode mudar no futuro.	Nicolas
D7	A Pi muda as taxas de envio (RF29); barramento e PIDs ficam fixos no firmware.	O ESP32 não depende da Pi para transmitir com segurança.	Isabela
D8	Saúde do ESP32 1 vez/s: versão, tempo ligado, último reinício, estado do OBD e erros.	Alimenta o RF37; a Isabela escolhe o que usar.	Isabela

#	Decisão	Por quê	Interessa a
D9	Mensagens binárias com CRC e número de sequência no cabo ESP32 → Pi.	3 a 4 vezes menores que JSON; nunca aceitam mensagem corrompida.	Isabela, Pedro
D10	O leitor da UART na Pi é da Isabela, com o decodificador do firmware.	Ela trabalha sem hardware e sem mexer em bytes.	Isabela
D11	Amostras perdidas não são reenviadas; sem a Pi escutando, o ESP32 descarta o que lê.	Sem a Pi não há evento a contextualizar; a sequência conta as perdas.	Isabela
D12	Horário pelo relógio do ESP32, convertido na Pi: intervalo exato, erro de até 10 ms.	A Pi sabe a hora pelo GPS (chrony com PPS).	Isabela, Nicolas
D13	No carro, ESP32 e Pi ligados por 4 fios (TX, RX, GND e 5 V), não por USB.	Soldados na placa, resistem à vibração.	Thiago, Nicolas
D14	O código é escrito e testado primeiro no notebook (fase 1), depois vai para o ESP32 e para a Pi (fases 2 e 3) e só então é integrado (fase 4).	Desenvolver e analisar sem hardware, sem carro e sem disputar a Pi.	Todos
D15	Após o ensaio no carro fica um ou outro: se o real funcionar, a simulação sai do firmware; senão, segue só a simulação. O simulador e a ECU simulada, no notebook, ficam.	O projeto entregue tem um único caminho; a simulação cobre o risco R06.	Todos
D16	Sem perda relevante (T24.2): em 2 h, menos de 0,1% de perdas, nenhuma mensagem corrompida aceita e nenhum reinício.	Torna o aceite mensurável.	Pedro
D17	PlatformIO com ESP-IDF, em C++; GPS e PPS ligados na Pi.	FreeRTOS, CAN e watchdog nativos; lógica testável no PC.	Todos
D18	Repositório V-guard-org/firmware_telemetria, decisões nas Issues, diagramas em Mermaid; docs em português e código em inglês.	Mesmo padrão dos repositórios visao e gerenciador_eventos.	Todos

#	Decisão	Por quê	Interessa a
D19	No notebook, o firmware roda com uma camada fina de plataforma: threads, CAN virtual do Linux e UART virtual no lugar do FreeRTOS, do CAN e da UART do ESP32.	A lógica testada é a mesma nas duas plataformas; na fase 2 só essa camada muda.	Todos
D20	A ECU simulada é um programa no notebook: fala no CAN virtual na fase 1 e, na fase 2, no barramento real pelo segundo ESP32, usado como adaptador USB-CAN.	Uma ECU só, escrita uma vez, para as duas fases.	Nicolas
D21	Na fase 2, o notebook conversa com o ESP32 pelo USB dele (IMU simulada e amostras); os fios só entram na fase 4.	Mesmo protocolo e mesmo decodificador; nada a comprar.	Thiago

Informações que faltam

Seis informações ainda dependem de outros membros. A mais urgente é o carro do ensaio, porque define a velocidade do barramento (D3).

- [ ] **Todos:** qual carro será usado no ensaio de 25/11 (T17.3)?
- [ ] **Todos:** quais modelos de IMU e de transceptor CAN serão comprados (T14.1, T10)?
- [ ] **Thiago e Nicolas:** o ESP32 recebe 5 V do pino da Pi ou de um ramo do conversor (T18)? Ele consome cerca de 0,1 A.
- [ ] **Thiago:** qual UART da Pi 4 recebe os fios? A principal é usada pelo Bluetooth, mas há UARTs extras por dtoverlay.
- [ ] **Thiago:** usar um transceptor com modo silencioso controlado pelo ESP32? A equipe do firmware recomenda, se uma pesquisa no datasheet confirmar.
- [ ] **Nicolas:** configurar o chrony com o PPS do GPS na Pi (T16), necessário para o erro de até 10 ms do D12.

Próximos passos

Depois da confirmação da Isabela, o desenvolvimento começa no notebook, sem depender de hardware.

1. A Isabela confirma os três pontos do início desta página.

2. Cinco pesquisas rápidas confirmam os fatos que faltam: barramento CAN de bancada com diodos entre dois ESP32, segundo ESP32 como adaptador USB-CAN, modo silencioso do transceptor, UARTs extras da Pi 4 e a ferramenta PlatformIO com ESP-IDF.
3. As questões técnicas restantes viram um mapa de decisões nas Issues do repositório novo no GitHub.
4. Fase 1, tudo no notebook, na ordem: comunicação e decodificador, simulador, OBD-II no CAN virtual e robustez. Depois, as fases 2 e 3 levam o mesmo código ao ESP32 e à Pi.
5. Na fase 4, ESP32 e Pi são integrados pelos fios. Datas fixas: integração em Nov 5, 2026 (T40), horários validados em Nov 11, 2026 (T41) e ensaio no carro em Nov 25, 2026 (T43.3).

Glossário

Estes termos têm um sentido só no firmware, na documentação e no código.

Termo	Significado	Evitar
Quadro CAN	Uma mensagem no barramento CAN do carro.	frame, pacote CAN
Requisição OBD	Quadro CAN padronizado (modo 01) que pede o valor de um PID; a única coisa que o firmware transmite no carro.	comando
PID	Identificador de um parâmetro do carro no OBD-II, como RPM ou velocidade.	sensor do carro
Mensagem	A unidade de comunicação entre o ESP32 e a Pi pelo cabo.	quadro, pacote
Fonte	A origem das amostras no ESP32: a IMU ou o OBD.	canal
Leitura	Medição bruta da IMU, interna ao ESP32; várias formam uma amostra.	amostra
Amostra	O que o ESP32 entrega à Pi: o estado de uma fonte num instante, com seu horário.	telemetria, registro
Amostra OBD	O conjunto completo dos valores OBD num instante, nunca um valor isolado.	leitura de PID
Telemetria consolidada	A linha por segundo que a Pi grava a partir das amostras.	—

Termo	Significado	Evitar
Evento	O registro que a Pi cria quando a visão detecta fadiga ou distração; o firmware nunca produz eventos.	alerta, ocorrência
Detecção	Uma condição percebida pelo firmware, como frenagem brusca (extensão futura); a Pi decide se vira evento.	evento
Simulação	O substituto temporário das fontes reais, usado só no desenvolvimento.	modo de teste
ECU simulada	O programa no notebook que responde às requisições OBD como se fosse o carro, no CAN virtual ou no barramento real.	simulador OBD
Adaptador USB-CAN	O segundo ESP32, que liga o notebook ao barramento CAN real na bancada.	ECU simulada (para a placa)